

AI Gateway POC

Elasticsearch Context Engineering and MCP Tool Calling

Purpose: Demonstrate a Go-based LLM gateway that stores memory in Elasticsearch, retrieves context during inference, calls tools through MCP, and exposes observability through Kibana.

1. Executive Summary

This proof of concept replaces PostgreSQL persistence with Elasticsearch-backed document storage and retrieval. The gateway stores conversations, messages, and inference logs as Elasticsearch documents. During inference, it builds a model prompt from recent conversation history, relevant retrieved context, and optional MCP tool results.

The current POC validates the core platform pattern: Elasticsearch acts as conversational memory and retrieval infrastructure, while MCP provides standardized access to external tools such as file reading, directory listing, or future business-system integrations.

2. Architecture

Component	Role	Current Implementation
Go Gateway	Receives API requests, builds context, calls the LLM provider, and streams or returns responses.	cmd/gateway with Chi routes
Elasticsearch	Stores conversations, messages, inference logs, and supports context retrieval.	Indexes: conversations, messages, inference_logs
Redis	Supports rate limiting and short-lived response caching.	Existing Redis service
MCP Server	Exposes external tools through a standard protocol.	Built-in test server under cmd/testmcp
LLM Provider	Generates assistant responses from assembled prompt context.	OpenRouter or Anthropic provider abstraction
Kibana	Visual UI for inspecting Elasticsearch documents and logs.	Added as docker-compose service on port 5601

3. POC Demo Flow

1. Start Elasticsearch, Redis, Kibana, and the Go gateway.
2. Create a conversation using POST /conversations.

3. Send a memory-setting inference request, for example: My preferred search engine is Elasticsearch. Remember this.
4. Ask a follow-up question such as: What search engine do I prefer?
5. Verify that the assistant answers from stored context.
6. Create a file under C:/tmp and call the MCP read_file tool through /infer.
7. Open Kibana at http://localhost:5601 and inspect messages, inference_logs, and raw_metadata.mcp.

4. Working Capabilities

- Conversation creation and retrieval through the existing REST API.
- Message persistence in Elasticsearch.
- Inference log persistence in Elasticsearch with provider metadata and token usage.
- Context assembly from recent messages and Elasticsearch full-text retrieval.
- MCP tool listing through GET /mcp/tools.
- Direct MCP tool calls through POST /mcp/tools/call.
- Inline MCP calls during inference using tool:<name> JSON arguments.
- Kibana visibility for conversations, messages, inference logs, and MCP metadata.
- SSE streaming remains available for inference responses.

5. Validation Evidence

The POC has already demonstrated successful context recall. After storing the statement that Elasticsearch is the preferred database, a later inference request asked which database was preferred, and the assistant answered that it was Elasticsearch.

The MCP path has also been validated. A request using tool:read_file with C:/tmp/mcp-test.txt returned an assistant response containing the file content: Hello from MCP server. This confirms that the gateway detected the inline tool call, invoked the MCP server, injected the result into the prompt, and saved the assistant response.

6. Feature Development Roadmap

Priority	Feature	Why It Matters
P0	AUTH_ENABLED and RATE_LIMIT_ENABLED toggles	Allows local testing without removing production security code.
P0	Persistent MCP sessions	Avoids starting a new MCP process per request and improves latency/reliability.
P1	Automatic tool planner	Allows users to ask naturally while the gateway decides whether to call MCP tools.
P1	Context deduplication	Prevents the current message or

		repeated retrieved text from appearing twice in prompts.
P1	Vector search with embeddings	Improves semantic retrieval beyond keyword matching.
P2	Tool allow/deny policy	Restricts risky tools and supports safer production deployments.
P2	Kibana dashboards	Adds ready-made views for latency, tokens, errors, and MCP usage.
P2	Provider-native tool schemas	Bridges MCP tools into provider-native tool/function calling where supported.

7. Production Considerations

- Enable Elasticsearch security, API keys, snapshots, and production index lifecycle policies.
- Run Elasticsearch with replicas in a multi-node environment. Yellow health is expected only in local single-node development.
- Store secrets outside .env for deployed environments.
- Add request tracing and correlation IDs across gateway, MCP tool calls, and provider responses.
- Add tests around repository behavior, MCP tool errors, prompt construction, and inference logging.

8. Recommended Next Sprint

8. Add environment toggles for authentication and rate limiting.
9. Convert MCP from per-request process startup to persistent managed sessions.
10. Add an LLM-based tool planner for automatic MCP selection.
11. Deduplicate retrieved context and add prompt-size controls.
12. Add Elasticsearch vector embeddings for semantic context retrieval.
13. Create a Kibana dashboard export for demo and operations.